

UNIDAD 8:

ENTRENAMIENTO SUPERVISADO

CURSO

IA

```
01 import openai
02 from langchain import
03     ChatOpenAI
04
05 def generar_respuesta(prompt):
06     modelo = ChatOpenAI(
07         model = "gpt-4o"
08         temperature = 0.7
09     )
10
11     respuesta = modelo.invoke(
12         prompt
13     )
14     return respuesta.content
15
16 if __name__ == "__main__":
17     prompt = "Explicame qué
18     es la inteligencia artificial"
19     print(generar_respuesta(prompt))
```



Docente:
Gabriel Mosso



BRAINSTORM

Rep.Argentina 786, Sunchales, Sta fe.



GabrielNMosso@GMail.com



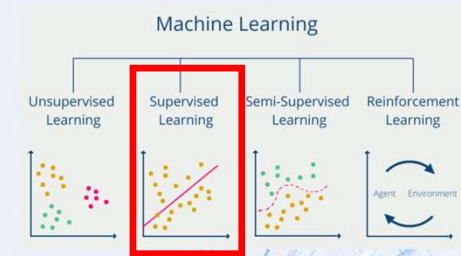
Tel: +54 351 3733383



ENTRENAMIENTO SUPERVISADO

El **entrenamiento supervisado** es una de las formas más comunes y fundamentales de enseñar a una inteligencia artificial.

Se basa en el uso de **datos etiquetados** o sea cada ejemplo que se le muestra al modelo ya viene con la **respuesta correcta**.



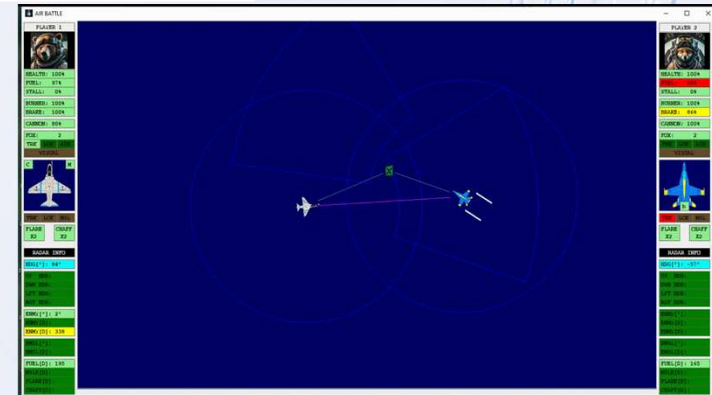
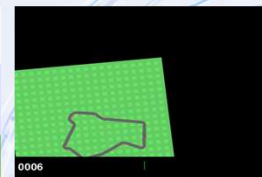
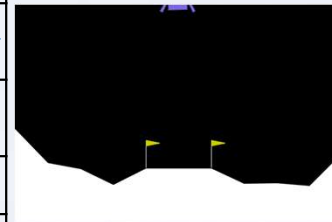
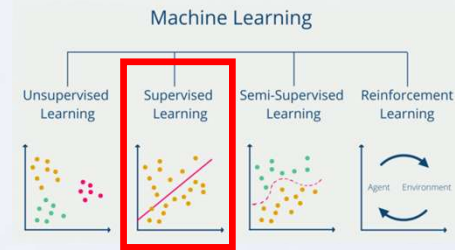
Aspecto	Descripción
Definición	Método de entrenamiento basado en datos etiquetados, donde cada entrada tiene una salida conocida.
Funcionamiento	- Se entregan ejemplos con entradas y salidas correctas. - El modelo aprende a relacionar entradas con salidas. - Se evalúa con datos nuevos.
Usos principales	- Clasificación (asignar categorías). - Regresión (predecir valores numéricos).
Ejemplos	- Detectar spam en correos electrónicos. - Predecir el precio de una casa.- Reconocimiento de imágenes.
Requisitos	- Gran cantidad de datos etiquetados. - Etiquetas consistentes y precisas. - Datos representativos del problema.
Ventajas	- Buen rendimiento si los datos son adecuados. - Evaluación objetiva. - Ideal para problemas concretos con resultados esperados.
Desventajas	- Costoso en tiempo y recursos por la necesidad de etiquetado. - Poco flexible con datos no etiquetados. - Riesgo de sobreajuste si no se controla bien.





ENTRENAMIENTO SUPERVISADO: GIMNASIOS DISPONIBLES

Nombre	Plataforma	Tipo de entornos	Ventajas destacadas	URL oficial / Repositorio
Gymnasium (ex OpenAI Gym)	Python	Juegos simples, control clásico, Atari	Modular, ligero, comunidad activa, fácil de integrar con algoritmos estándar	https://gymnasium.farama.org
DeepMind Control Suite	Python + MuJoCo	Control motor, robótica	Física realista, útil para benchmarking	https://github.com/deepmind/dm_control
Unity ML-Agents	Unity + Python	Juegos 3D, simulaciones complejas	Gráficos 3D, multijugador, integración con Deep Learning	https://unity.com/products/machine-learning-agents
PyBullet	Python	Simulación robótica	Gratuito, física precisa, alternativa libre a MuJoCo	https://pybullet.org/ / https://github.com/bulletphysics/bullet3
MuJoCo	Python	Simulación avanzada de robots	Alta fidelidad, usado en investigación avanzada	https://mujoco.org
PettingZoo	Python	Multiagente (juegos y simulaciones)	Diseñado para múltiples agentes, integración con Gym y Stable-Baselines	https://www.pettingzoo.ml
Meta-World	Python + MuJoCo	Manipulación robótica multitarea	Reutilización de conocimiento entre tareas, entorno realista de manipulación	https://github.com/rlworkgroup/metaworld
MineRL	Python + Java	Minecraft	Ambiente naturalista, tareas jerárquicas, integración con aprendizaje profundo	https://minerl.io/ / https://github.com/minerllabs/minerl
BrainStorm Air Battle	Python + Java	Juego 2D Combate	Desarrollo de estrategias, combate, supervivencia.	

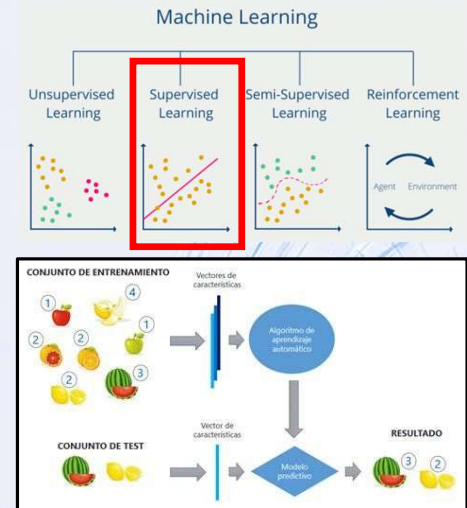




ENTRENAMIENTO SUPERVISADO [IMPLEMENTACION HIBRIDA]

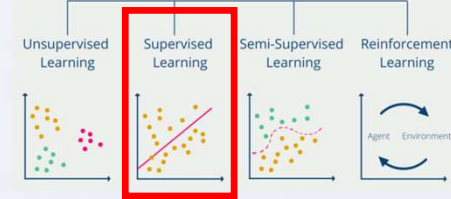
Etapa Inicial	Objetivo	Tareas específicas	Herramientas / Técnicas comunes	Errores frecuentes
1. Crear dataset manual	Generar un dataset inicial con ejemplos correctos de juego	Jugar partidas manualmente y registrar los estados (X) y comandos (Y)	CSV, pandas, grabación de partidas	Registrar casos incorrectos; Datos desbalanceados; pocas partidas para generalizar
2. Entrenar modelo inicial	Que la DNN aprenda a imitar el comportamiento humano	Definir arquitectura de la DNN, compilar, usar .fit() con dataset	TensorFlow/Keras, .fit(), pérdida binary_crossentropy	Overfitting por pocas partidas; aprender patrones erróneos si los datos son malos

Etapa Cíclica	Objetivo	Tareas específicas	Herramientas / Técnicas comunes	Errores frecuentes
3. Ciclo de auto-juego y reentrenamiento	Refinar progresivamente la política del agente	Subdividido en pasos cíclicos		
3.1 Generar partidas automáticas	Que la DNN explore y produzca nuevas experiencias	Ejecutar partidas con el modelo actual; almacenar estados (X) y acciones (Y)	model.predict(), entorno AirBattle, listas o arrays para registrar episodios	Dataset demasiado repetitivo; falta de exploración de nuevas estrategias
3.2 Modificar dataset con reglas	Mejorar dataset aplicando heurísticas de desempeño	Revisar los estados y acciones; aplicar reglas programadas (ej: HEALTH<0.2 → CMD_BRAKE=1)	NumPy, Python, reglas if-else, reward shaping	Aplicar reglas incorrectas; sobre-corriger acciones; sesgo excesivo
3.3 Reentrenar modelo con dataset modificado	Ajustar la política de la DNN basándose en nuevas experiencias	Cargar modelo con load_model y entrenar con .fit() usando el dataset modificado	TensorFlow/Keras, .fit(), model.save/load	No normalizar entradas; entrenar con demasiadas pocas o demasiadas épocas; olvidar guardar modelo
3.4: Evaluación de métricas del modelo	Analizar qué tanto aprendió el modelo y cuáles entradas (inputs) tienen más impacto sobre las decisiones (outputs).	detectar variables inútiles, correlaciones falsas o comportamientos inesperados antes de lanzar otra ronda de autogeneración de partidas.	Análisis de impacto, Pesos Efectivos, Correlación Inputs outputs.	Confundir métricas de entrenamiento (loss, accuracy) con análisis de pesos, Interpretación equivocada de pesos, Omitir verificación con lógica del dominio.





Machine Learning



MANAGER ENTRENAMIENTO

El objetivo del script unificador es **orquestrar todas las etapas** para que el entrenamiento sea cíclico y autónomo:

- **Cargar el modelo inicial o crear uno nuevo.**
- **Generar partidas automáticas** (el modelo juega contra sí mismo).
- **Recolectar datasets** (ganador/perdedor, jugador 1/jugador 2).
- **Modificar y filtrar datasets** aplicando las reglas del dominio.
- **Reentrenar el modelo** con los datos actualizados.
- **Evaluar métricas** (impacto de entradas, correlaciones, pesos).
- **Guardar el modelo actualizado.**

Repetir el ciclo N veces o hasta que se cumplan criterios de mejora.

Funciones principales del MANAGER:

El script debería estructurarse en **módulos o funciones**, cada una llamando a los scripts ya existentes:

- `generar_partidas()` → llama al script que simula partidas y genera datasets.
- `modificar_datasets()` → procesa datasets y aplica reglas de consistencia.
- `entrenar_modelo()` → entrena o reentrena con el dataset combinado.
- `evaluar_metricas()` → ejecuta los scripts de análisis (impacto, correlación, pesos).
- `guardar_modelo()` → versiona el modelo (ej: `model_v2.h5`).

El **manager** las ejecuta en orden dentro de un bucle for ciclo in range(N):

Errores frecuentes en la unificación:

- **Rutas mal configuradas:** cada script puede estar guardando datasets en carpetas distintas; el manager debe unificar rutas.
- **Sobrescribir datasets o modelos:** si no se versionan correctamente, se pierden resultados previos.
- **No controlar la convergencia:** repetir ciclos sin analizar si realmente hay mejora puede llevar a sobreentrenamiento o estancamiento.
- **Falta de logs:** sin guardar un historial de métricas por ciclo, es imposible comparar la evolución.
- **Exceso de automatización:** no dejar puntos de control para revisión humana puede hacer que el modelo aprenda errores acumulados.

Training Room Bot Room Data/Analysis

Flu	Name	Gen	Flt	Games	Win	Draw	Loss	Diff	Stall	Turns	Break	FOX	Win	Can	Flt	Chaff	Mal	Fuel	Mal	Flt	Chaff	Mal	Fuel
000	2023.11.30-20.36.03	010	10	53	72	65	45	17	0	1.0	1	1	1	0	22	9	1	1	0.04	1	0	0	0
001	2023.12.01-05.55.54	015	07	111	66	62	64	20	0	1.0	1	1	1	0	28	9	1	1	0.10	1	0	0	0
002	2023.11.30-21.23.23	012	02	177	59	56	52	28	0	0.0	0	0	1	0	21	8	0	0	0.05	1	0	0	0
003	2023.11.30-23.39.33	014	04	142	61	61	61	22	0	1.0	1	1	1	0	25	8	1	1	0.07	1	0	0	0
004	2023.12.01-05.47.02	015	02	154	61	62	52	25	0	0.0	0	0	1	1	20	7	1	1	0.10	1	0	0	0
005	2023.11.30-17.01.43	005	06	101	64	56	48	20	0	0.0	0	0	1	1	23	9	0	0	0.06	1	0	0	0
006	2023.11.30-23.44.41	014	06	162	65	63	56	23	0	0.0	0	0	1	1	29	9	1	1	0.11	1	0	0	0
007	2023.11.30-23.53.54	014	09	159	57	61	59	25	0	1.0	1	1	1	0	27	7	2	2	0.06	1	0	0	0
008	2023.12.01-05.53.37	015	05	156	58	65	63	23	0	0.0	0	0	1	1	29	10	1	0	0.08	1	0	0	0
009	2023.11.30-18.09.38	006	08	204	59	57	49	23	0	1.0	1	1	1	0	24	9	1	1	0.06	1	0	0	0
010	2023.11.30-20.33.53	010	09	143	60	59	49	20	0	0.0	0	0	1	1	23	7	0	0	0.10	1	0	0	0
011	2023.11.30-21.29.16	012	04	140	56	59	59	22	0	1.0	1	1	1	0	23	8	1	1	0.11	1	0	0	0
012	2023.12.01-02.49.02	017	07	132	60	68	60	19	0	1.0	1	1	1	0	28	11	2	1	0.04	1	0	0	0
013	2023.11.30-20.29.33	010	07	149	57	53	50	22	0	0.0	0	0	1	0	22	8	1	1	0.16	1	0	0	0
014	2023.11.30-21.20.27	012	01	152	56	55	54	24	0	0.0	0	0	1	0	24	6	1	1	0.09	1	0	0	0
015	2023.12.01-04.57.42	019	08	126	56	65	60	25	0	0.0	0	0	1	1	29	9	2	2	0.14	1	0	0	0
016	2023.02.12-15.35.12	046	56	67	64	15	1	2.0	0	5	1	1	1	27	11	0	0	0.05	0	0	0	0	0
017	2023.02.12-15.26.18	103	56	63	61	19	0	1.0	1	7	1	1	1	30	10	1	1	0.11	0	0	0	0	0
018	2023.02.13-05.51.33	90	60	69	65	17	0	0.0	2	6	1	1	1	32	11	2	2	0.06	1	0	0	0	0
019	0-075-F-10-2025.02.13-20.50.27	070	10	51	59	68	64	17	2	1.0	3	9	1	0	27	10	2	2	0.12	1	1	0	0
020	020 2025.02.13-09.18.33	76	58	65	65	16	0	0.0	2	6	1	1	1	27	11	2	2	0.09	0	0	0	0	0
021	0-077-F-09-2025.02.16-04.04.08	077	09	31	71	73	52	15	0	0.0	2	8	1	0	31	11	2	2	0.06	2	0	0	0
022	0-077-F-10-2025.02.16-04.01.29	077	10	30	67	75	60	12	0	0.0	2	7	1	0	29	12	1	1	0.23	1	0	0	0
023	2025.02.12-22.15.39	82	56	70	64	16	1	0.0	0	7	2	1	1	29	10	1	1	0.07	2	0	0	0	0
024	024 2023.11.30-17.10.03	005	05	176	56	54	45	21	0	2.0	2	1	1	25	9	0	0	0.07	1	0	0	0	0
025	0-080-F-09-2025.02.16-07.19.18	080	08	15	10	89	59	8	0.9	1	8	2	1.9	33	14	2	2	0.20	2	1	0	0	0
026	026 2023.11.30-20.47.46	014	07	154	55	65	65	19	0	0.0	0	0	1	1	31	7	1	2	0.05	1	0	0	0
027	0-075-F-10-2025.02.15-13.40.52	075	10	32	53	59	64	16	1	1.0	1	7	2	1	32	9	0	0	0.16	2	0	0	0
028	028 2023.11.30-19.31.58	009	10	190	54	67	48	20	0	0.0	0	0	1	1	25	8	0	0	0.10	1	0	0	0
029	0-075-F-09-2025.02.16-04.05.22	075	09	21	67	62	55	11	0	0.0	2	7	1	1	26	11	2	1	0.14	1	0	0	0
030	0-074-F-04-2025.02.16-06.37.38	074	04	37	54	61	53	20	1	0.0	1	7	1	0	25	9	1	1	0.11	1	0	0	0
031	0-075-F-07-2025.02.15-13.44.15	075	07	42	55	64	54	17	1	1.0	2	0	1	1	31	10	2	2	0.07	2	0	0	0
032	0-080-F-07-2025.02.16-07.15.47	080	07	9	67	73	49	15	0	0.0	2	8	1	1	16	11	2	2	0.17	1	0	0	0
033	0-044-F-09-2025.02.13-14.32.37	044	08	75	54	70	63	21	1	0.0	0	6	2	1	24	14	1	0	0.17	0	0	0	0
034	0-080-F-09-2025.02.16-07.22.42	080	09	10	60	45	42	30	1	0.0	1	9	1	0.2	24	7	2	2	0.20	1	0	0	0
035	0-081-F-10-2025.02.16-09.22.11	081	10	5	60	47	57	24	2	0.0	1	8	1	0	26	9	2	2	0.20	2	0	1	0
036	0-075-F-06-2025.02.16-04.55.17	075	06	27	63	59	55	17	0	0.0	1	6	0	0.1	27	10	1	1	0.11	1	0	0	0
037	0-081-F-06-2025.02.16-09.11.49	081	06	6	67	65	53	14	0	0.2	2	6	0	0	20	11	2	1	0.08	0	0	0	0
038	0-081-F-07-2025.02.16-09.14.24	081	07	6	50	59	50	12	0	0.0	1	8	1	0	28	10	1	1	0.08	2	0	0	0
039	0-075-F-09-2025.02.16-04.57.53	075	09	23	63	56	46	14	0	0.0	1	6	2	1.9	20	9	1	2	0.17	1	0	0	0
040	0-081-F-09-2025.02.16-09.17.00	081	09	9	56	49	55	24	0	0.0	1	7	0	0	31	9	1	1	0.09	2	0	0	0



Etapa 1: Crear Dataset Manual

Generar un dataset inicial que contenga ejemplos correctos de juego, es decir, pares **estado** → **acción** donde la acción tomada sea razonablemente buena. Este dataset servirá como base para entrenar la red neuronal antes de que aprenda a jugar por sí misma.

Tareas específicas:

- Jugar partidas manualmente en AirBattle.
- Asegurarse de que los datos cubran distintos escenarios: atacar, esquivar, recolectar ítems, etc.
- Crear DataSet unificado.

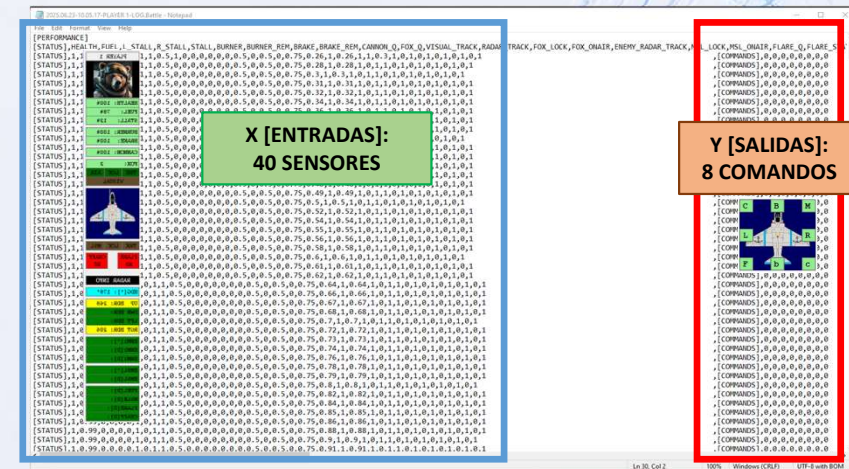
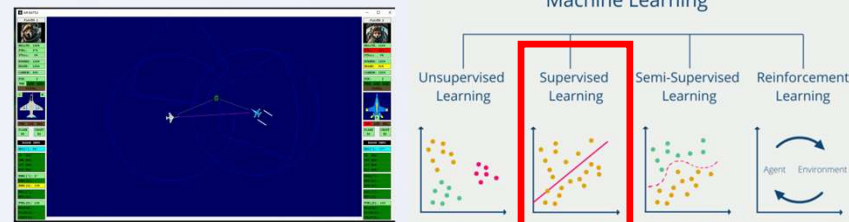
Herramientas / Técnicas comunes:

- CSV o archivos Excel para almacenar datos.
- Pandas para manipular y limpiar los datasets.
- Scripts en Python para convertir logs del juego en tablas de estado y comandos.

Errores frecuentes:

- **Pocas partidas para generalizar:** El dataset solo refleja tus decisiones en un tipo de escenario, la red no aprende situaciones diversas.
- **Sesgo del jugador:** Si siempre juegas agresivo o pasivo, la DNN puede aprender una estrategia limitada.
- **Duplicación de datos:** Filas repetidas que sobre-representan ciertos estados y distorsionan el aprendizaje.
- **Falta de exploración de casos extremos:** No incluir situaciones críticas (salud baja, enemigos cercanos, pocos recursos) que son importantes para que la DNN aprenda a reaccionar.
- **Falta de diversidad de escenarios:** Si todas las partidas ganadas se obtienen con la misma estrategia, la DNN no aprenderá a adaptarse a situaciones distintas.

Machine Learning



```
SCRIPT >>> "TRAIN02_UNIFICADOR_DATASET.py"
```



Etapla 2: Entrenar modelo inicial

Que la DNN aprenda a **imitar las decisiones correctas de las partidas manuales**, utilizando los datasets generados por AirBattle (ganador y perdedor). Esta etapa establece la **política inicial** del agente antes de que comience a jugar contra sí mismo y a refinarse.

Tareas específicas:

Definir la arquitectura de la DNN:

Entradas: 40 sensores del avión.

Capas ocultas: p.ej., 2 capas densas con 10 neuronas cada una.

Salidas: 8 comandos (CMD_LEFT, CMD_RIGHT, ...).

Preparar los datos:

Leer los CSV de ganador y perdedor.

Normalizar las entradas (HEALTH, FUEL, etc.) para que los valores estén en rangos compatibles con la función de activación (sigmoid).

Opcional: ponderar los ejemplos de ganadores más alto que los perdedores para reforzar buenas acciones.

Compilar el modelo:

Seleccionar **optimizador** (adam, sgd).

Seleccionar **función de pérdida** adecuada (binary_crossentropy) porque las salidas son acciones binarias.

Entrenar con .fit():

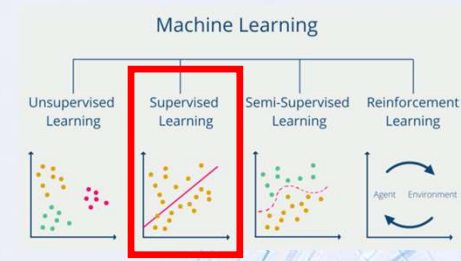
Ajustar el modelo usando el dataset manual.

Definir número de épocas, tamaño de batch y validación si es necesario.

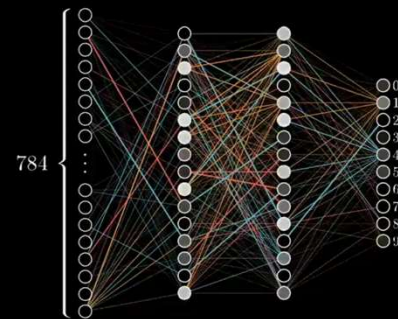
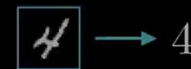
Guardar el modelo entrenado:

Para poder cargarlo en etapas posteriores y continuar el entrenamiento iterativo.

```
model.save("airbattle_model_initial.h5")
```



Training in progress. . .



```
SCRIPT >>> "TRAIN03_AIRBATTLE_ENTRENAMIENTO.py"
```



Etapa 2: Entrenar modelo inicial

Que la DNN aprenda a **imitar las decisiones correctas de las partidas manuales**, utilizando los datasets generados por AirBattle (ganador y perdedor). Esta etapa establece la **política inicial** del agente antes de que comience a jugar contra sí mismo y a refinarse.

Errores frecuentes

Overfitting por pocas partidas:

La DNN memoriza patrones específicos de las partidas manuales y no generaliza bien a nuevas situaciones.

Dataset desequilibrado:

Si hay muchos más ejemplos de perdedor que de ganador, la DNN puede aprender estrategias incorrectas.

No normalizar las entradas:

Valores como HEALTH o FUEL en escalas muy diferentes pueden hacer que la DNN aprenda lentamente o mal.

Arquitectura inadecuada:

Muy pocas neuronas/capas → incapaz de aprender patrones complejos.

Muy grande → entrenamiento lento y riesgo de overfitting.

Funciones de activación incorrectas:

Usar relu en la capa de salida para acciones binarias puede producir resultados no binarios; se recomienda sigmoid o softmax según el enfoque.

Batch o número de épocas inapropiado:

Demasiadas épocas → sobreajuste; pocas épocas → aprendizaje incompleto.

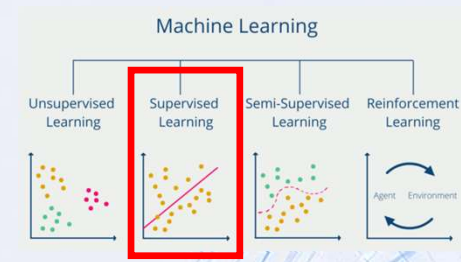
Tamaño de batch demasiado grande → poca variabilidad por actualización; demasiado pequeño → entrenamiento ruidoso.

No validar durante entrenamiento:

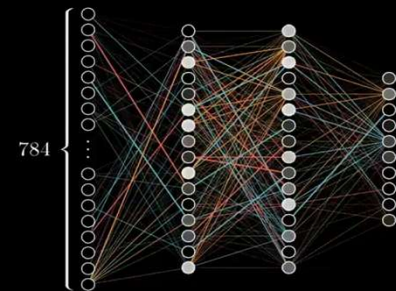
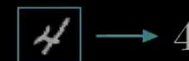
Sin un dataset de validación, no se detecta si el modelo está aprendiendo correctamente o simplemente memorizando ejemplos.

Ignorar secuencia temporal:

.fit() trata cada fila como independiente; si el juego tiene secuencia de estados, la DNN podría no capturar ciertas dinámicas temporales (esto se corrige más adelante con reglas o RL).



Training in progress...



```
SCRIPT >>> "02-AIRBATTLE-GO-ENTRENAMIENTO.py"
```

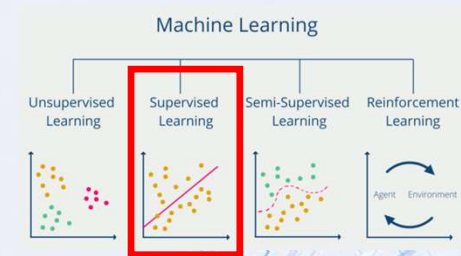



Etapla 3.1: Generar partidas automáticas

Permitir que la DNN juegue contra sí misma para **producir nuevas experiencias** que luego se usarán para mejorar la política del agente. Esto permite que la red aprenda a enfrentarse a situaciones más diversas que las observadas en las partidas manuales.

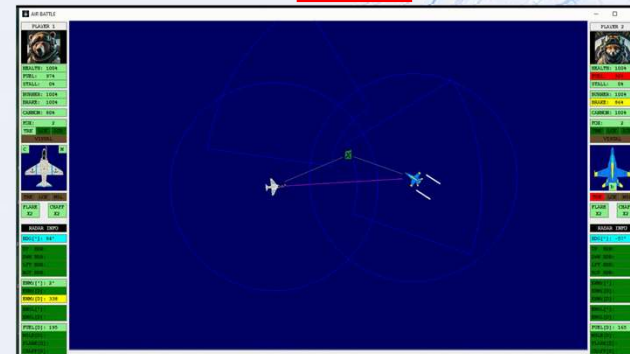
Tareas específicas:

- **Cargar el modelo entrenado inicialmente:** Se usa el modelo entrenado en la Etapa 2.
- **Configurar el entorno AirBattle:**
 - Inicializar la partida para que ambos jugadores sean controlados por la DNN.
 - Registrar cada frame los estados (X) y las acciones tomadas (Y).
- **Ejecutar la partida automáticamente:**
 - En cada paso del juego, predecir las acciones usando `model.predict(state)`.
 - Aplicar un umbral (p.ej., >0.5) para decidir si activar cada comando.
 - Guardar estados y comandos en arrays o listas.
- **Al finalizar la partida:** AirBattle genera dos datasets automáticamente: uno para el **ganador** y otro para el **perdedor**.



Errores frecuentes

- **Dataset demasiado repetitivo:** Si la DNN juega siempre igual, los estados y acciones generados no aportan información nueva.
- **Falta de exploración:** La DNN puede quedarse atrapada en estrategias subóptimas ("política local"), porque solo replica lo aprendido.
- **Desalineación de estados y acciones:** Guardar un estado con la acción de un frame diferente puede corromper el dataset.
- **Errores de sincronización con el juego:** Latencia o retrasos en la predicción pueden generar acciones aplicadas tarde o inconsistentes.
- **Sobrepeso de un jugador:** Si uno de los jugadores DNN es consistentemente más fuerte, el dataset del perdedor podría ser demasiado amplio y no reflejar situaciones útiles.
- **Fallos en la predicción de la DNN:** Salidas continuas sin binarizar correctamente los comandos pueden generar datos incorrectos.
- **Archivos corruptos o no guardados:** Partida interrumpida por error del juego o fallo de disco puede provocar datasets incompletos.
- **Ignorar diversidad de escenarios:** Si siempre se inicializa la partida de la misma manera, los datos generados no cubren todas las posibles situaciones del juego.



```
SCRIPT >>> "TRAIN01_EJECUTADOR_AIR_BATTLE.py"
```

```
SCRIPT >>> "COMMS01_KB_PLAYER.py"
```

```
SCRIPT >>> "COMMS02_AI_PLAYER.py"
```



Etapa 3.2: Modificar dataset con reglas

Objetivo:

Refinar el dataset generado por las partidas automáticas para **corregir errores, eliminar datos irrelevantes y aplicar reglas heurísticas** que mejoren la política de la DNN antes del reentrenamiento.

Tareas específicas:

Unificar datasets de partidas ganadoras:

- Recorrer la carpeta de archivos .Battle generados por AirBattle.
- Concatenar todos los archivos en un único dataset unificado (Dataset_Unificado.txt).

Depuración de datos:

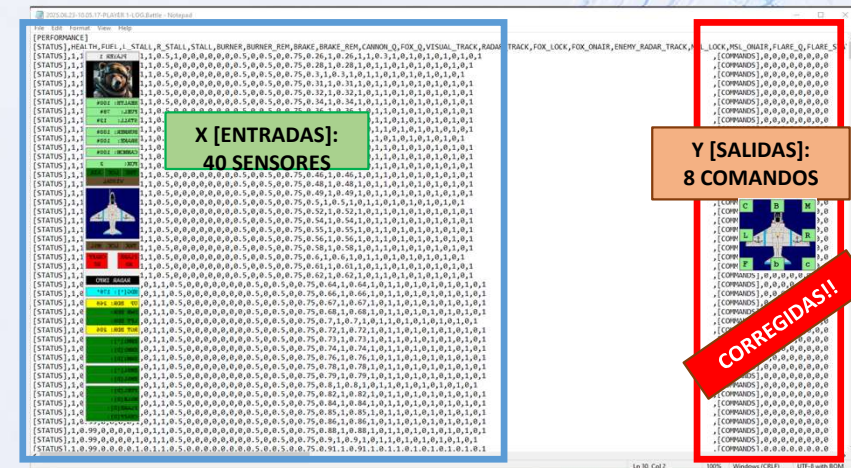
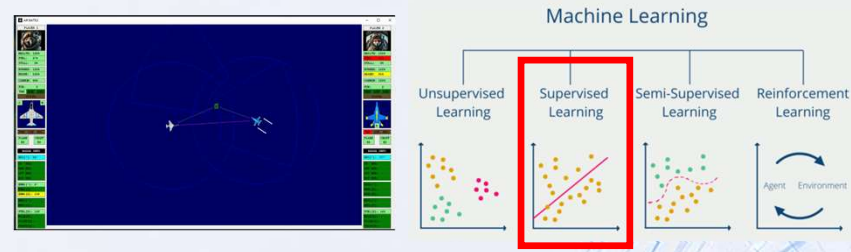
- Leer el dataset unificado y filtrar filas que **no aportan información**: por ejemplo, filas donde **todos los comandos están en cero**.
- Contar estadísticas de filas totales, descartadas y conservadas.

Aplicar reglas heurísticas sobre los datos:

- **Ejemplo 1:** Si $STALL > 0.7$, desactivar comandos de giro (CMD_LEFT y CMD_RIGHT) para evitar perder control del avión.
- **Ejemplo 2:** (ya incluido) filas con todos los comandos en cero se descartan automáticamente. Estas reglas **mejoran la calidad del dataset**, asegurando que la DNN no aprenda comportamientos peligrosos o inútiles.

Errores frecuentes en esta etapa

- **Reglas demasiado estrictas:** Aplicar correcciones excesivas puede eliminar variabilidad útil y limitar la capacidad de la DNN de aprender estrategias alternativas.
- **Descartar filas útiles:** Filas donde todos los comandos son cero podrían representar situaciones de espera estratégica; descartarlas siempre puede eliminar casos válidos.



SCRIPT >>> "TRAIN02_UNIFICADOR_DATASET.py"



Etapa 3.2: Modificar dataset con reglas

Action Masking

En un entorno, no siempre todas las acciones son válidas. *Action Masking* consiste en **inhibir o forzar a 0** la probabilidad de elegir acciones imposibles o inútiles en cierto estado.

Ejemplo en AirBattle (40 sensores, 8 comandos):

- Si **no tenés misiles disponibles**, entonces la acción *"Disparar misil"* se **enmascara** → probabilidad = 0.
- Si estás **al borde del mapa**, la acción *"avanzar más allá del límite"* gira hacia un lado.

El agente solo distribuye su exploración sobre acciones **válidas**, acelerando el aprendizaje y evitando que gaste pasos en decisiones sin sentido.

Ventajas:

- Reduce el espacio de búsqueda.
- Aumenta la eficiencia de entrenamiento.
- Evita que el agente aprenda a "castigarse" con acciones imposibles.

Label Smoothing

En vez de entrenar al modelo con etiquetas "duras" (ej. acción correcta = 1, demás = 0), se introduce **incertidumbre intencional** en las etiquetas.

Ejemplo clásico:

- Sin smoothing: acción correcta = 1.0, demás = 0.0.
- Con smoothing (ej. factor 0.1): acción correcta = 0.9, las demás = 0.1 distribuidas.

Ejemplo en AirBattle (supervisado / imitation learning):

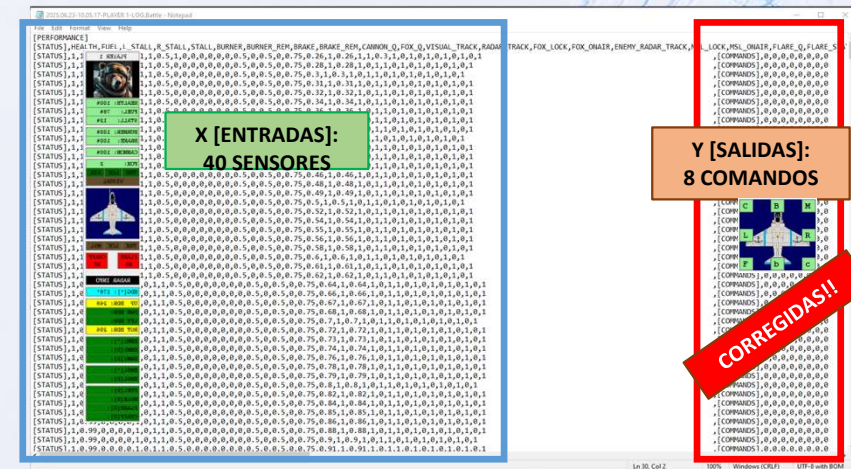
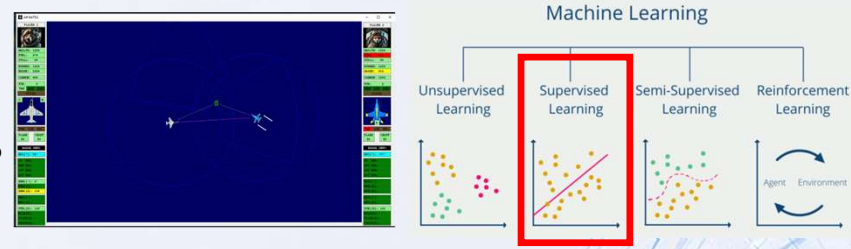
Si el humano en cierto estado dispara un misil (acción correcta = 1), el modelo entrenaría con:

Misil = **0.9**

Otras acciones (giro, chaff, etc.) = **0.1 / 7 ≈ 0.014** cada una.

Ventajas:

- Evita **sobreaajuste** a las etiquetas duras.
- Hace que la red sea más **robusta a ruido** o decisiones ambiguas.
- Mejora la **generalización** (no aprende que hay "una sola acción correcta" en todas las situaciones).



SCRIPT >>> "TRAIN02_UNIFICADOR_DATASET.py"



Etapas 3.3: Reentrenar modelo con dataset modificado

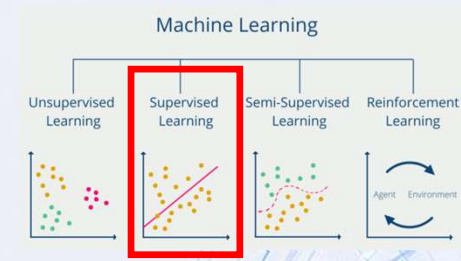
Usar el **dataset depurado y mejorado con reglas heurísticas** para volver a entrenar el modelo. El propósito es que la red neuronal aproveche **tanto la experiencia de las partidas automáticas como las correcciones impuestas por reglas humanas** para alcanzar un comportamiento más robusto y seguro en el entorno AirBattle.

Tareas específicas:

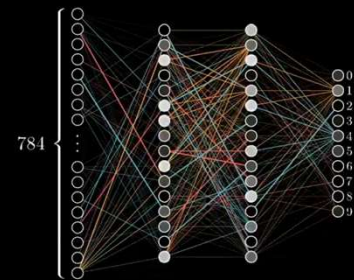
- **Cargar el nuevo dataset (Dataset_Depurado.txt):**
- **Reinicializar o cargar el modelo previo:** Cargar pesos anteriores y **seguir entrenando** para no perder el conocimiento acumulado.
- **Entrenar el modelo con el dataset modificado:** Ajustar batch_size, epochs y optimizador. Monitorear pérdida (loss) y métricas de precisión.
- **Guardar el modelo actualizado:** Exportar pesos y arquitectura en un nuevo archivo (modelo_reentrenado.h5).

Errores frecuentes en esta etapa

- **Sobrescribir el modelo previo sin respaldo:** Se pierde la comparación entre versiones (modelo inicial vs. modelo reentrenado).
- **Overfitting al dataset modificado:** El modelo memoriza las correcciones pero pierde generalización, especialmente si el dataset es pequeño.
- **Reglas mal aplicadas en 3.2 reflejadas en el modelo:** Si las reglas fueron demasiado estrictas o erróneas, el modelo aprenderá un comportamiento sesgado.
- **Olvidar probar en simulación real:** El modelo puede verse "perfecto" en métricas, pero en el juego real tomar decisiones inútiles.



Training in progress...



```
SCRIPT >>> "TRAIN03_AIRBATTLE_ENTRENAMIENTO.py"
```



Etapa 3.4: Evaluación de métricas del modelo

Analizar qué tanto aprendió el modelo y **cuáles entradas (inputs) tienen más impacto sobre las decisiones (outputs).**

Esto permite detectar **variables inútiles, correlaciones falsas o comportamientos inesperados** antes de lanzar otra ronda de autogeneración de partidas.

Tareas específicas:

Medir impacto de las entradas (6-ANALISIS IMPACTO.py):

- Se extraen los **pesos de la primera capa** de la red neuronal.
- Permite ver **qué inputs (ej. FUEL, STALL, ENEMY_RADAR_TRACK)** son más determinantes en la propagación hacia adelante.
- Error típico:** confundir magnitud de pesos con importancia absoluta (a veces inputs normalizados cambian la interpretación).

Analizar pesos efectivos Input-Output (8-PESOS EFECTIVOS INPUTS-OUTPUTS.py):

- Se multiplican los pesos de todas las capas para aproximar el “camino” de influencia entre cada input y cada output.
- Permite generar una **matriz de relevancia Input → Output**.
- Error típico:** interpretar correlaciones espurias como “lógica aprendida”.

Calcular correlación estadística Input-Output (7-CORRELACION INPUTS-OUTPUTS.py):

- Se miden coeficientes de correlación (Pearson o Spearman) entre las columnas de entradas y las salidas predichas.
- Diferencia con los pesos: aquí no se mide la red, sino la **relación estadística en los datos y predicciones**.
- Error típico:** usar correlación lineal en fenómenos no lineales → se pierden relaciones útiles.

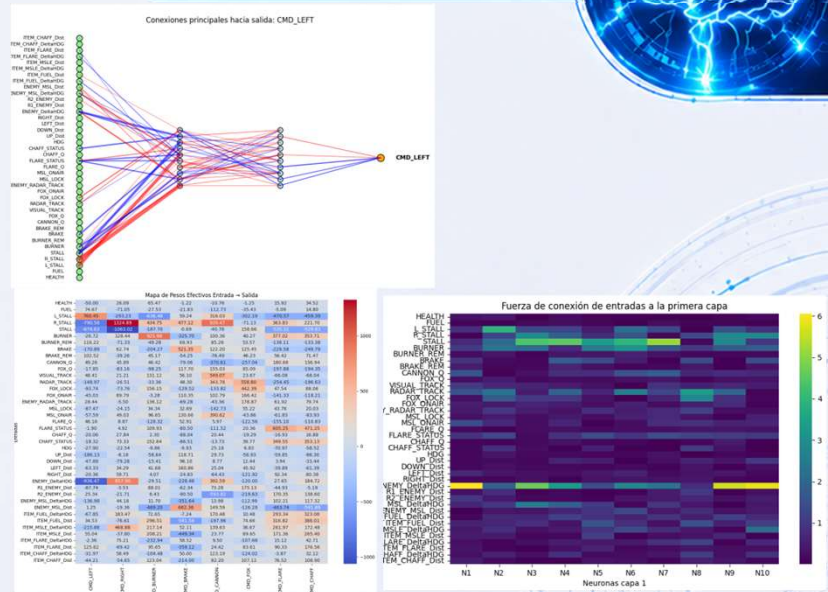
Errores frecuentes en esta etapa

Confundir métricas de entrenamiento (loss, accuracy) con análisis de pesos: Una buena accuracy no significa que el modelo esté usando los inputs correctos.

Interpretación equivocada de pesos: Pesos grandes no siempre significan más importancia (puede depender de la normalización de datos).

Omitir verificación con lógica del dominio: Se puede **aprender patrones espurios o irrelevantes** que están en los datos, aunque no tengan sentido desde el punto de vista del sistema real.

No comparar con versiones anteriores: La gracia de esta etapa es ver cómo cambió el modelo respecto al inicial y al reentrenado.



SCRIPT >>> "CHECK01_PESOS_EFECTIVOS_IO.py"

SCRIPT >>> "CHECK02_CORRELACION_IO.py"

SCRIPT >>> "CHECK03_GRAFICO_CONEXIONES.py"

SCRIPT >>> "CHECK04_ANALISIS_IMPACTO_INPUTS.py"

SCRIPT >>> "CHECK05_CAMINOS_EFECTIVOS.py"

SCRIPT >>> "CHECK06_CAMINOS_EFECTIVOS_IMPORT.py"



```
> print("Gracias !")
> print("dudas, consultas...")
> input("Presione una tecla para salir...")
```



Docente:
Gabriel Mosso



BRAINSTORM

Rep. Argentina 786, Sunchoales, Sta fe.



GabrielNMosso@GMail.com



Tel: +54 351 3733383

